CU Boulder / Coursera

Spacecraft Dynamics Capstone Project

Attitude Dynamics and Control of a Nano-Satellite Orbiting Mars

Course Report

Student: Haniyeh Zahra Budaqi
Instructor: Prof. Dr. Hanspeter Schaub

May 2026

Contents

Abstract

In this project, the knowledge acquired from the Spacecraft Dynamics and Control specialization is applied to perform attitude control of a nano-satellite orbiting Mars while operating as a daughter-craft to a mother spacecraft in a geostationary Mars orbit (GMO). Through the completion and analysis of eleven project tasks, an attitude control architecture is developed and validated. The resulting simulation enables three operational modes based on the spacecraft mission requirements and relative orbital geometry:

1) Science mode: pointing the spacecraft sensor toward the Martian surface,

2) Power mode: orienting the solar panels toward the Sun,

3) Communication mode: pointing the communication antenna toward the mother spacecraft.

The spacecraft dynamics, reference-frame generation, attitude error evaluation, and closed-loop control simulations are implemented in MATLAB, while mission visualization is performed using AGI Systems Tool Kit (STK).

Chapter 1

Introduction

This project considers a small science satellite orbiting Mars in a Low Mars Orbit (LMO), tasked with collecting scientific data from the Martian surface and relaying the acquired information to a larger mother spacecraft orbiting in a Geostationary Mars Orbit (GMO). In addition to science and communication operations, the spacecraft must periodically orient its solar panels toward the Sun to maintain sufficient onboard power.

Accordingly, the spacecraft mission is divided into three operational modes:

1) **Science Mode:** pointing the sensor toward the Martian surface,
2) **Power Mode:** orienting the solar panels toward the Sun,
3) **Communication Mode:** pointing the communication antenna toward the GMO spacecraft.

Both the LMO and GMO spacecraft are assumed to follow known circular orbits whose orientations are described using Hill reference frames generated through a 3-1-3 Euler-angle sequence.

The LMO spacecraft is equipped with:

- A science sensor aligned with the $+b_1$ body axis,
- Solar panels whose normal vector aligns with the b_3 axis,
- A communication antenna aligned with the $-b_1$ axis.

The objective of this project is to design and validate an attitude determination and control framework capable of autonomously switching between the required operational pointing modes.

Chapter 2

Mission Scenario Definitions

2.1 Mission Orbit Overview

The LMO and GMO spacecraft are both assumed to follow circular orbits around Mars. The orbital motion is modeled using classical two-body dynamics, where Mars is treated as the central gravitational body.

The physical and orbital parameters used throughout the simulations are summarized in Table 2.1.

Table 2.1: Simulation and Spacecraft Parameters

Parameter	Value	Unit
Mars radius R_{Mars}	3396.19	km
Mars gravitational parameter μ_{Mars}	4.28283×10^{13}	m^3/s^2
LMO altitude	400	km
LMO orbital radius r_{LMO}	3796.19	km
GMO orbital radius r_{GMO}	20424.2	km
LMO orbital angular rate $\dot{\theta}_{LMO}$	0.000884797	rad/s
GMO orbital angular rate $\dot{\theta}_{GMO}$	0.0000709003	rad/s

The orbital angular rates are computed using the circular-orbit relation:

$$\dot{\theta} = \sqrt{\frac{\mu_{Mars}}{r^3}}$$

The initial orbit orientation angles of both spacecraft are summarized in Table 2.2.

Table 2.2: Initial Orbit Frame Orientation Angles

Spacecraft	Ω (deg)	i (deg)	$\theta(t_0)$ (deg)
LMO	20	30	60
GMO	0	0	250

Figure 2.1 illustrates the definition of a circular orbit using relative elements along with inertial and Hill frames' base vectors. Note that the orbital elements (Ω, i, θ) can be used as (3-1-3) Euler Angles to obtain Hill frame $\mathcal{H}$ from inertial frame $\mathcal{N}$.

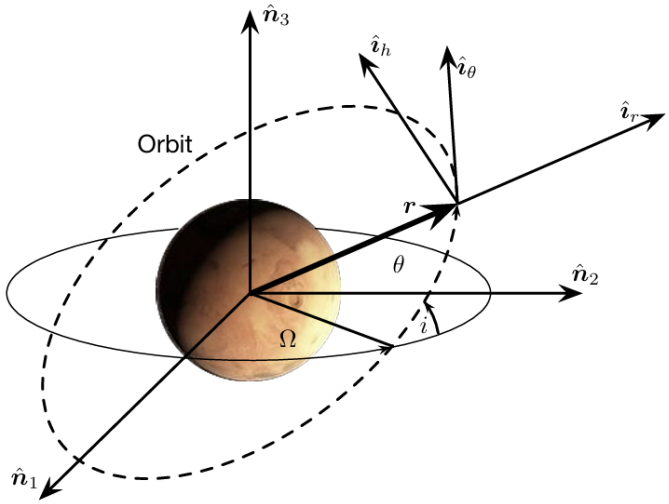


Figure 2.1: Illustration of circular orbital elements alongside inertial frame $\mathcal{N}$: $\{\hat{\boldsymbol{n}}_1, \hat{\boldsymbol{n}}_2, \hat{\boldsymbol{n}}_3\}$ and Hill frame $\mathcal{H}$: $\{\hat{\boldsymbol{i}}_r, \hat{\boldsymbol{i}}_\theta, \hat{\boldsymbol{i}}_h\}$ base vectors

2.2 Mission Pointing Scenarios

A body frame as shown in Figure 2.2 is mounted on the spacecraft, assuming to be the principle body frame.

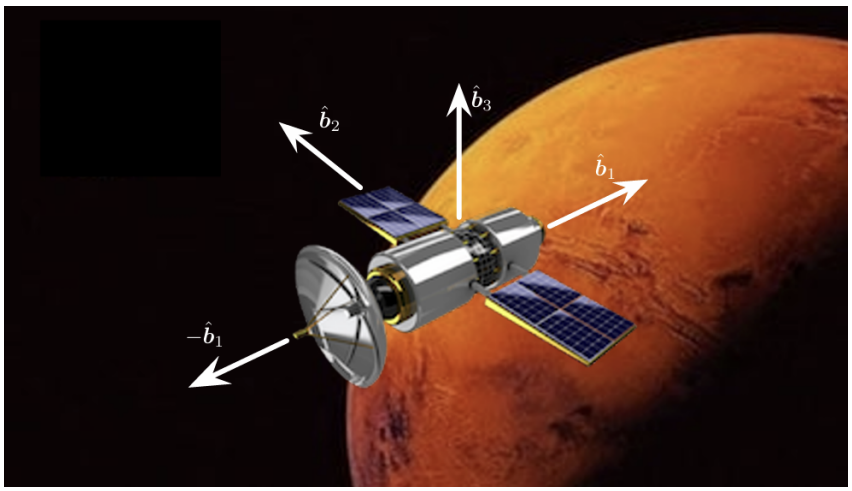


Figure 2.2: Illustration of LMO spacecraft's body frame $\mathcal{B}$: $\{\hat{\boldsymbol{b}}_1, \hat{\boldsymbol{b}}_2, \hat{\boldsymbol{b}}_3\}$

Three operational spacecraft pointing modes are defined according to mission requirements.

The Sun is assumed to be located at an effectively infinite distance and aligned with the inertial n_2 axis. Therefore, during Sun-pointing operation, the spacecraft solar-panel axis b_3 must align with the inertial n_2 direction.

The spacecraft operational modes are summarized as follows:

- **Power Mode:** active whenever the spacecraft is on the sunlit side of Mars,
- **Science Mode:** active during eclipse when GMO communication is unavailable,
- **Communication Mode:** active whenever the GMO spacecraft lies within the communication visibility constraint.

The communication mode is activated when the angular separation between the LMO and GMO position vectors is less than 35°.

Table 2.3: Mission Pointing Scenario Summary

Orbital Situation	Operational Mode
Spacecraft on sunlit Mars side	Power Mode
Spacecraft in eclipse and GMO visible	Communication Mode
Spacecraft in eclipse and GMO not visible	Science Mode

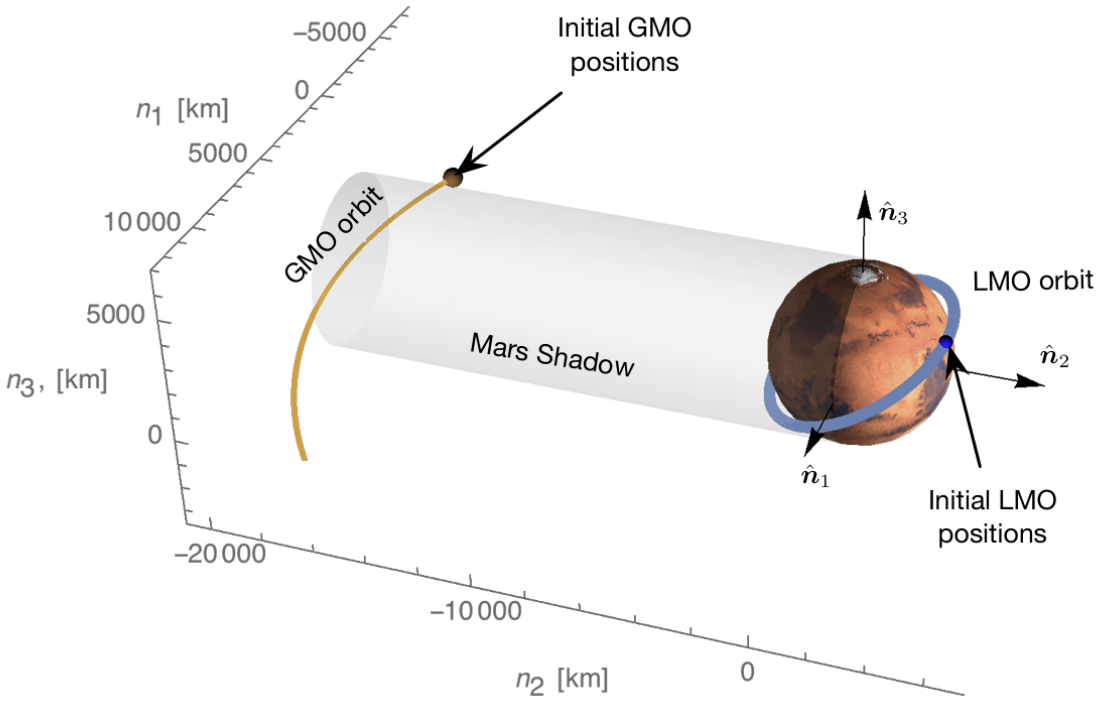


Figure 2.3: Mission Orbit Scenario Illustration

2.3 Spacecraft Attitude States

The spacecraft initial attitude is represented using Modified Rodrigues Parameters (MRPs), while the initial angular velocity is expressed in the spacecraft body frame.

The initial spacecraft attitude is defined as:

$$\boldsymbol{\sigma}_{B/N}(t_0) = \begin{bmatrix} 0.3 \\ -0.4 \\ 0.5 \end{bmatrix}$$

The initial body angular velocity is:

$${}^{\mathcal{B}}\boldsymbol{\omega}_{B/N}(t_0) = \begin{bmatrix} 1.00 \\ 1.75 \\ -2.20 \end{bmatrix} deg/s$$

The rigid-body spacecraft inertia tensor is assumed to be:

$${}^{\mathcal{B}}[I] = \begin{bmatrix} 10 & 0 & 0 \\ 0 & 5 & 0 \\ 0 & 0 & 7.5 \end{bmatrix} \text{ kg}\cdot\text{m}^2$$

The spacecraft is assumed to be capable of generating arbitrary three-axis control torques through an onboard actuator system.

2.4 Attitude Control Law Overview

A proportional-derivative (PD) control law is used throughout this project to regulate spacecraft attitude motion:

$${}^{\mathcal{B}}\mathbf{u} = -K\boldsymbol{\sigma}_{B/R} - P\,{}^{\mathcal{B}}\boldsymbol{\omega}_{B/R}$$

where:

- $\boldsymbol{\sigma}_{B/R}$ is the attitude tracking error,
- ${}^{\mathcal{B}}\boldsymbol{\omega}_{B/R}$ is the angular velocity tracking error,
- K and P are controller gain matrices.

The objective of the controller is to detumble the spacecraft while driving the body frame toward a desired mission reference frame.

2.5 Implementation Methods

[NEED REVISION BEFORE EVEN BEING WRITTEN]

Chapter 3

Task Descriptions and Solutions

3.1 Orbits

3.1.1 Task 1: Orbit Simulation

Assume the general orbit frame as $\mathcal{O} : \{\hat{i}_r, \hat{i}_\theta, \hat{i}_h\}$. Even though the general orbit frame and Hill frame basically use the same base vectors, their difference is found in their origin. $\mathcal{O}$ frame has its origin aligned with the origin of inertial frame $\mathcal{N}$ – which is the center of Mars. The Hill frame $\mathcal{H}$, however, is mounted on the spacecraft, therefore has the same origin as the body frame $\mathcal{B}$.

Both these frames are obtainable through a (3-1-3) Euler Angle rotation on the inertial frame, though the hill frame origin is also translated to the spacecraft.

Now, using the $\mathcal{O}$ frame, spacecraft position vector is expressed as:

$$^{\mathcal{O}}\boldsymbol{r} = r\hat{\boldsymbol{i}}_r$$

Both LMO and GMO orbits are circular, therefore having the same angular velocity $\dot{\theta}$ all over the orbit. Spacecraft velocity vector is expressed as:

$$^{\mathcal{O}}\dot{\boldsymbol{r}} = r\dot{\theta}\hat{\boldsymbol{i}}_\theta$$

The MATLAB function `orbitTheta()` takes time, angular velocity and initial angle to calculate the element θ using the following equation:

$$\theta_t = \dot{\theta}t + \theta_0$$

```
1 function theta = orbitTheta(time,thetaDot,theta0)
2
3 theta = rem(thetaDot*time+theta0, 2*pi);
4
5 end
```

Taking the position and velocity vectors from the general orbit frame to the inertial frame is done using the DCM $[NO]$ which can be obtained having all orbital elements at a given time.

$$[ON] = M_3(\theta_t)M_1(i)M_3(\Omega)$$

$$[NO] = [ON]^T$$

$${}^{\mathcal{N}}\boldsymbol{r} = [NO]\,{}^{\mathcal{O}}\boldsymbol{r}$$

$${}^{\mathcal{N}}\dot{\boldsymbol{r}} = [NO]\,{}^{\mathcal{O}}\dot{\boldsymbol{r}}$$

This is achieved by the MATLAB function `orbit_state()` as follows:

```
function [r,r_dot] = orbit_state(r,Omega,i,theta,theta_dot)

% Since the motion is planar and circular, at any time we
   have
% r_dot = 0 i_r + r*theta_dot i_theta + 0 i_h

r_O = [r;0;0];
r_dot_O = [0;r*theta_dot;0];

ON = M3(theta)*M1(i)*M3(Omega);
NO = transpose(ON);

% Taking the O frame vectors to the N frame
r = (NO*r_O) / 1000;
r_dot = (NO*r_dot_O) / 1000;

% r parameters are expressed in km
% r_dot parameters are expressed in km/s

end
```

3.1.2 Task 2: Orbit Frame Orientation

The Hill-frame basis vectors were constructed directly from the inertial position and velocity vectors:

$${}^{\mathcal{N}}\hat{\boldsymbol{i}}_r = \frac{\boldsymbol{r}_{LMO}}{\|\boldsymbol{r}_{LMO}\|}$$

$$^{\mathcal{N}}\hat{\boldsymbol{i}}_h = \frac{\boldsymbol{r}_{LMO} \times \dot{\boldsymbol{r}}_{LMO}}{\|\boldsymbol{r}_{LMO} \times \dot{\boldsymbol{r}}_{LMO}\|}$$

$$^{\mathcal{N}}\hat{\boldsymbol{i}}_\theta = \hat{\boldsymbol{i}}_h \times \hat{\boldsymbol{i}}_r$$

To drive a DCM with the base vectors in either frames, we have:

$$[FB] = \begin{bmatrix} ^{\mathcal{F}}\hat{\boldsymbol{b}}_1 & ^{\mathcal{F}}\hat{\boldsymbol{b}}_2 & ^{\mathcal{F}}\hat{\boldsymbol{b}}_3 \end{bmatrix} = \begin{bmatrix} ^{\mathcal{B}}\hat{\boldsymbol{f}}_1^T \\ ^{\mathcal{B}}\hat{\boldsymbol{f}}_2^T \\ ^{\mathcal{B}}\hat{\boldsymbol{f}}_3^T \end{bmatrix}$$

Therefore, to find $[HN]$ we will have:

$$[HN] = \begin{bmatrix} ^{\mathcal{N}}\hat{\boldsymbol{i}}_r^T \\ ^{\mathcal{N}}\hat{\boldsymbol{i}}_\theta^T \\ ^{\mathcal{N}}\hat{\boldsymbol{i}}_h^T \end{bmatrix}$$

A MATLAB function named `orbit_frame_dcm(t)` was implemented to evaluate the time-varying Hill-frame orientation.

```
function HN_DCM = orbit_frame_dcm(t)
h_dc = 400*1000; % m
R_mars = 3396.19*1000; % m
r_LMO = R_mars + h_dc; % m

mio_mars = 42828.3*10^9; % m^3/s^2
theta_dot_LMO = sqrt(mio_mars / (r_LMO^3));  % rad/s

RA_dc = 20 * pi/180; % DC right ascention
i_dc = 30 * pi/180;  % DC Inclination
theta_dc_0 = 60 * pi/180; % DC initial angle

theta_dc_t = rem(theta_dot_LMO*t + theta_dc_0, 2*pi);
[r_dc_t, r_dot_dc_t] = orbit_state(r_LMO, RA_dc, i_dc,
   theta_dc_t, theta_dot_LMO);

i_r = r_dc_t / norm(r_dc_t);
i_h = cross(r_dc_t, r_dot_dc_t) / norm(cross(r_dc_t,
   r_dot_dc_t));
```

```
18
19 i_theta = cross(i_h, i_r);
20
21 HN_DCM = [i_r';i_theta';i_h'];
22 end
```

3.2 Reference Frame Orientation

3.2.1 Task 3: Sun-Pointing Reference Frame Orientation

To point the spacecraft solar panels axis $\hat{\boldsymbol{b}}_3$ at the Sun, the reference frame $\mathcal{R}_s$ must be chosen such that $\hat{\boldsymbol{r}}_3$ axis points in the Sun direction ($\hat{\boldsymbol{n}}_2$ in the scenario). To define the frame completely, the first axis $\hat{\boldsymbol{r}}_1$ is assumed to point in $-\hat{\boldsymbol{n}}_1$ direction. Therefore, the base vectors will be:

$$\hat{\boldsymbol{r}}_1 = -\hat{\boldsymbol{n}}_1$$

$$\hat{\boldsymbol{r}}_3 = \hat{\boldsymbol{n}}_2$$

$$\hat{\boldsymbol{r}}_2 = \hat{\boldsymbol{r}}_3 \times \hat{\boldsymbol{r}}_1 = -(\hat{\boldsymbol{n}}_2 \times \hat{\boldsymbol{n}}_1) = \hat{\boldsymbol{n}}_3$$

Similar to the previous task, the analytical expression for the DCM can be obtained using the base vectors:

$$[R_s N] = \begin{bmatrix} {}^{\mathcal{N}}\hat{\boldsymbol{r}}_1^T \\ {}^{\mathcal{N}}\hat{\boldsymbol{r}}_2^T \\ {}^{\mathcal{N}}\hat{\boldsymbol{r}}_3^T \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

The resulting reference-frame DCM can also be obtained using a (3-1-3) Euler Angle of $(\pi, \frac{\pi}{2}, 0)$ rotation on it. MATLAB function `RsN_DCM()` serves this purpose:

```
1 function rotation = RsN_DCM()
2
3 rotation = M1(pi/2)*M3(pi);
4
5 end
```

Since the base vectors of this frame are just a re-arrangement of the inertial frame base vectors, the frame remains fixed relative to the inertial frame, thus:

$$\boldsymbol{\omega}_{R_s/N} = 0$$

3.2.2 Task 4: Nadir-Pointing Reference Frame Orientation

To point the spacecraft sensor platform axis $\hat{\boldsymbol{b}}_1$ towards the center of Mars (nadir direction), the reference frame $\mathcal{R}_n$ must be chosen such that $\hat{\boldsymbol{r}}_1$ axis points towards the planet. To completely describe the frame, we assume the second axis $\hat{\boldsymbol{r}}_2$ points in velocity direction $\hat{\boldsymbol{i}}_\theta$. Therefore, we'll have:

$$\hat{\boldsymbol{r}}_1 = -\hat{\boldsymbol{i}}_r$$

$$\hat{\boldsymbol{r}}_2 = \hat{\boldsymbol{i}}_\theta$$

$$\hat{\boldsymbol{r}}_3 = \hat{\boldsymbol{r}}_1 \times \hat{\boldsymbol{r}}_3 = -\hat{\boldsymbol{i}}_h$$

This frame can be obtained using a (3-1-3) Euler Angles rotation on the Hill frame with angles $(\pi, \pi, 0)$.

$$[R_n H] = M_1(\pi) M_3(\pi)$$

We also had previously obtained $[HN]$ in task 2 as a function of time. So,

$$[R_n N](t) = [R_n H][HN](t)$$

In Matlab, this task is done by function `RnN_DCM()`.

```
function rotation = RnN_DCM(t)

RnH = M1(pi)*M3(pi);
HN = orbit_frame_dcm(t); % Function from task 2

rotation = RnH*HN;
end
```

Since this frame is a re-arrangement of the Hill frame, it also moves with it, thus:

$$\boldsymbol{\omega}_{R_n/N} = \boldsymbol{\omega}_{H/N} = \dot{\theta}\hat{\boldsymbol{i}}_\theta$$

3.2.3 Task 5: GMO-Pointing Reference Frame Orientation

To point the nano-satellite communication platform axis $\hat{\boldsymbol{b}}_1$ towards the GMO mother spacecraft, the communication reference frame $\mathcal{R}_c$ must be chosen such that $-\hat{\boldsymbol{r}}_1$ axis points towards the GMO satellite location, defining $\Delta\boldsymbol{r} = \boldsymbol{r}_{GMO} - \boldsymbol{r}_{LMO}$. So,

$$\hat{\boldsymbol{r}}_1 = -\frac{\Delta\boldsymbol{r}}{\|\Delta\boldsymbol{r}\|}$$

Further, to fully define a three-dimensional reference frame, assume the second axis is defined as

$$\hat{\boldsymbol{r}}_2 = \frac{\Delta \boldsymbol{r} \times \hat{\boldsymbol{n}}_3}{\|\Delta \boldsymbol{r} \times \hat{\boldsymbol{n}}_3\|}$$

Naturally,

$$\hat{\boldsymbol{r}}_3 = \hat{\boldsymbol{r}}_1 \times \hat{\boldsymbol{r}}_2$$

Having all three base vectors in the inertial frame, we have:

$$[R_c N] = \begin{bmatrix} {}^{\mathcal{N}}\hat{\boldsymbol{r}}_1^T \\ {}^{\mathcal{N}}\hat{\boldsymbol{r}}_2^T \\ {}^{\mathcal{N}}\hat{\boldsymbol{r}}_3^T \end{bmatrix}$$

The MATLAB function `RcN_DCM(t)` is written to deliver this task.

```
function rotation = RcN_DCM(t)

h_dc = 400*1000;                                % m
R_mars = 3396.19*1000;                          % m
r_LMO = R_mars + h_dc;                          % m
mio_mars = 42828.3*10^9;                        % m^3/s^2
theta_dot_LMO = sqrt(mio_mars / (r_LMO^3));     % rad/s
RA_dc = 20 * pi/180;                            % DC right
   ascention
i_dc = 30 * pi/180;                             % DC
   Inclination
theta_dc_0 = 60 * pi/180;                       % DC initial
   angle
r_GMO = 20424.2*1000;                           % m
theta_dot_GMO = sqrt(mio_mars / (r_GMO^3));     % rad/s

RA_mc = 0;
i_mc = 0;
theta_mc_0  = 250 * pi/180;              % MC initial angle

theta_dc_t  = orbitTheta(t, theta_dot_LMO, theta_dc_0);
[r_dc_t, ~] = orbit_state(r_LMO, RA_dc, i_dc, theta_dc_t,
   theta_dot_LMO);

theta_mc_t  = orbitTheta(t, theta_dot_GMO, theta_mc_0);
[r_mc_t, ~] = orbit_state(r_GMO, RA_mc, i_mc, theta_mc_t,
```

```
   theta_dot_GMO);

n3 = [0;0;1];
dr = r_mc_t - r_dc_t;
r1 = -dr/norm(dr);   % Since -r1 should point toward the
   mother craft
r2 = cross(dr,n3)/norm(cross(dr,n3));
r3 = cross(r1,r2);

RcN = [r1';r2';r3'];
rotation = RcN;

end
```

To derive the relative angular velocity ${}^{\mathcal{N}}\boldsymbol{\omega}_{R_c/N}$, we'll use an equation (3.27) in book give reference later :

$$[\dot{C}] = -[\tilde{\boldsymbol{\omega}}][C]$$

Therefore,

$$[\tilde{\boldsymbol{\omega}}_{R_c/N}] = \begin{bmatrix} 0 & -\omega_3 & \omega_2 \\ \omega_3 & 0 & -\omega_1 \\ -\omega_2 & \omega_1 & 0 \end{bmatrix} = -[\dot{R_c N}][R_c N]^T$$

To derive the rate of a DCM numerically, we'll have:

$$[\dot{R_c N}] \approx \frac{[R_c N]_{t+dt} - [R_c N]_t}{dt}$$

This algorithm is implemented in MATLAB function `RcN_omega(t)`. The choice of different `dt` will result in different precisions in finding the angular velocity.

```
function w_RcN = RcN_omega(t)

dt = 1;
RcN = RcN_DCM(t);
RcN_next = RcN_DCM(t+dt);

RcN_dot = (1/dt)*(RcN_next - RcN);

w_tilde = -RcN_dot*transpose(RcN);

w1 = w_tilde(3,2);
```

```
w2 = w_tilde(1,3);
w3 = w_tilde(2,1);

w_RcN = [w1;w2;w3];
end
```

3.3 Attitude Evaluation and Simulator

3.3.1 Task 6: Attitude Error Evaluation

Any attitude feedback control algorithm requires the ability to compute the attitude and angular velocity tracking errors of the current body frame $\mathcal{B}$ relative to the reference frame $\mathcal{R}$. To find a set of tracking errors $\begin{bmatrix} \boldsymbol{\sigma}_{B/R} \\ {}^{\mathcal{B}}\boldsymbol{\omega}_{B/R} \end{bmatrix}$, we have the known variables $\boldsymbol{\sigma}_{B/N}$, ${}^{\mathcal{B}}\boldsymbol{\omega}_{B/N}$, as well as the desired reference frame DCM $[RN]$ and frame rate ${}^{\mathcal{N}}\boldsymbol{\omega}_{R/N}$. We first need to drive the DCM $[BN]$ which can be obtained from $\boldsymbol{\sigma}_{B/N}$ using equation (3.152):

$$[C] = [I_{3\times3}] + \frac{8[\tilde{\boldsymbol{\sigma}}]^2 - 4(1-\sigma^2)[\tilde{\boldsymbol{\sigma}}]}{(1+\sigma^2)^2}$$

The opposite of this operation–converting a DCM to its MRP form–is formulated in equation (3.153):

$$[\tilde{\boldsymbol{\sigma}}] = \frac{[C]^T - [C]}{\zeta(\zeta+2)}$$

$$\zeta = \sqrt{trace([C]) + 1}$$

These two forms are implemented in MATLAB function `sigmaToDCM()` and `DCMToSigma()` in respect.

Now, the roadmap to finding the tracking errors would be:

$$\boldsymbol{\sigma}_{B/N} \rightarrow [BN]$$

$$[BR] = [BN][NR] = [BN][RN]^T$$

$$[BR] \rightarrow \boldsymbol{\sigma}_{B/R}$$

$${}^{\mathcal{B}}\boldsymbol{\omega}_{B/R} = {}^{\mathcal{B}}\boldsymbol{\omega}_{B/N} - {}^{\mathcal{B}}\boldsymbol{\omega}_{R/N} = {}^{\mathcal{B}}\boldsymbol{\omega}_{B/N} - [BN]\,{}^{\mathcal{N}}\boldsymbol{\omega}_{R/N}$$

A MATLAB function named `attitudeErrorEval()` was implemented to evaluate both attitude and angular velocity tracking errors.

```
function [sigma_BR, w_BR] = attitudeErrorEval(sigma_BN, w_BN,
    RN, w_RN)
```

```
2
3 BN = sigmaToDCM(sigma_BN);
4 BR = BN*transpose(RN);
5
6 sigma_BR = DCMToSigma(BR);
7 w_BR = w_BN - BN*w_RN;
8
9 end
```

3.3.2 Task 7: Numerical Attitude Simulator

In order to integrate the attitude we need a numerical integrator. The LMO and GMO orbit satellite positions are already calculated in earlier tasks. Let the propagated attitude state $\boldsymbol{X}$ be

$$\boldsymbol{X} = \begin{bmatrix} \boldsymbol{\sigma}_{B/N} \\ {}^{\mathcal{B}}\boldsymbol{\omega}_{B/N} \end{bmatrix}$$

Assuming that the spacecraft is rigid and there's no external torque acting on it, its dynamics obey

$$[I]\dot{\boldsymbol{\omega}}_{B/N} = -[\tilde{\boldsymbol{\omega}}_{B/N}][I]\boldsymbol{\omega}_{B/N} + \boldsymbol{u}$$

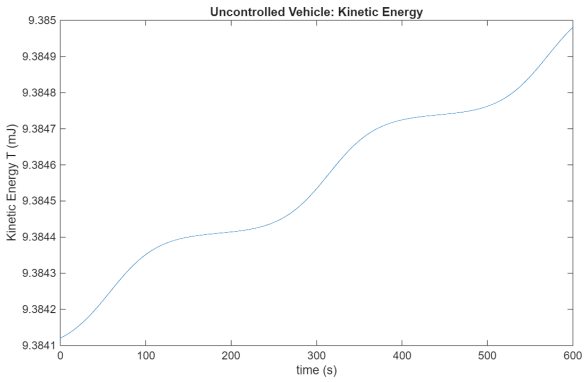
where $\boldsymbol{u}$ is the external control torque vector.

For this task, a simple loop integrator was written in MATLAB (which comparing to RK4, proved to be more time-consuming yet more precise). The control torques $\boldsymbol{u}_1 = 0$ and ${}^{\mathcal{B}}\boldsymbol{u} = (0.01, -0.01, 0.02)$ were applied respectively and the following were computed:

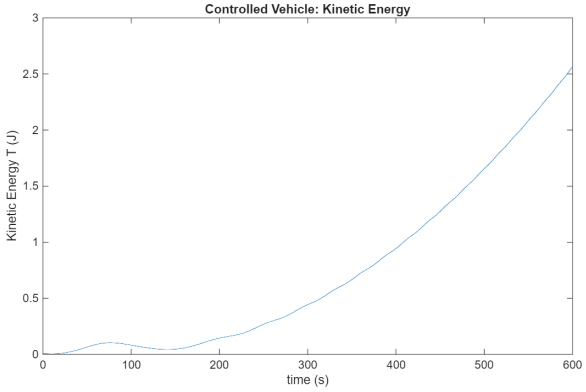
- Attitude MRP $\boldsymbol{\sigma}_{B/N}$
- Angular velocity $\boldsymbol{\omega}_{B/N}$
- Angular momentum vector $\boldsymbol{H} = [I]\boldsymbol{\omega}_{B/N}$ in both $\mathcal{N}$ and $\mathcal{B}$ frames
- Rotational kinetic energy $T = \frac{1}{2}\boldsymbol{\omega}_{B/N}^T[I]\boldsymbol{\omega}_{B/N}$

The results of both simulations are plotted in Figure 3.1. As is seen, after applying the control torque $\boldsymbol{u}_2$ on the spacecraft, its rotational kinetic energy plummets (from approximately 9 mJ in uncontrolled mode (Figure 3.1a) to more than 2.5 J when controlled (Figure 3.1b)). This is verified by spacecraft angular velocity, whose 1st and 2nd components start to oscillate in controlled mode while its 3rd goes through a rapid, non-stop increase (Figure 3.1f).

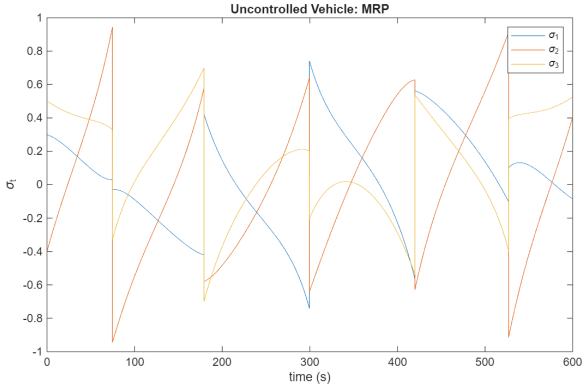
The MRP confirms it all. The spacecraft goes crazily oscillating when the control torque $\boldsymbol{u}_2$ is applied to it (Figure 3.1d). This suggests that $\boldsymbol{u}_2$ fails as a successful control, making the craft uncontrollable rather that detumbling it.



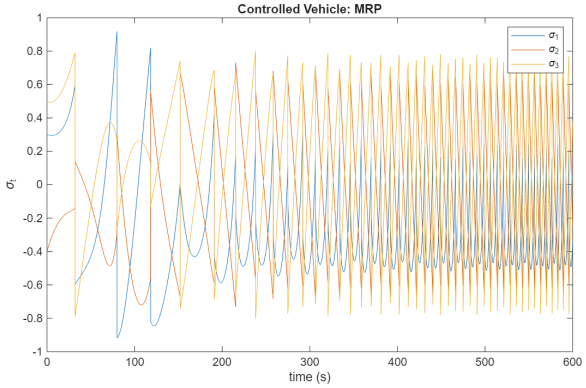
(a) $\boldsymbol{u}_1$ Applied: T



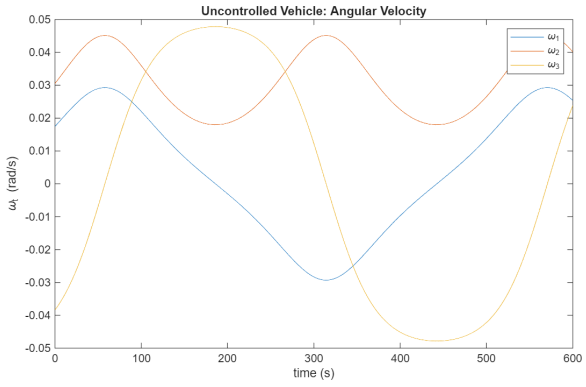
(b) $\boldsymbol{u}_2$ Applied: T



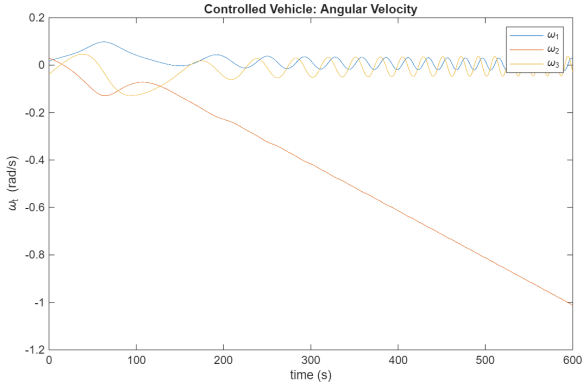
(c) $\boldsymbol{u}_1$ Applied: $\boldsymbol{\sigma}_{B/N}$



(d) $\boldsymbol{u}_2$ Applied: $\boldsymbol{\sigma}_{B/N}$



(e) $\boldsymbol{u}_1$ Applied: $\boldsymbol{\omega}_{B/N}$



(f) $\boldsymbol{u}_2$ Applied: $\boldsymbol{\omega}_{B/N}$

Figure 3.1: Results of the simulations in task 7

3.4 Completing the Mission

3.4.1 Task 8: Sun Pointing Control

A PD attitude controller was implemented to drive the spacecraft toward the Sun-pointing reference frame.

The control torque is defined as:

$$\mathbf{u} = -K\boldsymbol{\sigma}_{BR} - P\boldsymbol{\omega}_{BR}$$

The controller gains were selected using linearized closed-loop dynamics to satisfy desired transient-response requirements.

Simulation results demonstrated stable convergence toward the desired Sun-pointing orientation.

3.4.2 Task 9: Nadir Pointing Control

The nadir-pointing control mode used the same PD control architecture while continuously updating the rotating reference frame.

The controller successfully tracked the time-varying nadir reference orientation while maintaining stable spacecraft rotational dynamics.

3.4.3 Task 10: GMO Pointing Control

The GMO-pointing control mode continuously oriented the spacecraft communication platform toward the GMO spacecraft.

The communication reference frame and corresponding angular velocity were updated dynamically throughout the simulation.

Simulation results demonstrated stable closed-loop tracking of the moving communication reference frame.

3.4.4 Task 11: Mission Scenario Simulation

A complete mission simulation was developed to automatically switch between spacecraft operational modes according to mission conditions.

The mission logic was implemented as follows:

- If the spacecraft was illuminated by the Sun, Sun-pointing mode was activated,
- If the spacecraft was in eclipse and the GMO spacecraft was visible, communication mode was activated,

- Otherwise, nadir-pointing science mode was selected.

At every simulation step:

1. The active reference frame was selected,
2. The reference angular velocity was evaluated,
3. The attitude tracking errors were computed,
4. The PD control torque was generated,
5. The spacecraft rotational dynamics were propagated using RK4 integration.

The complete mission simulation demonstrated stable closed-loop attitude behavior and smooth transitions between operational spacecraft modes.

3.5 Simulation Results and Discussion

Simulation results demonstrated stable spacecraft attitude behavior under all operational modes. The implemented proportional-derivative controller successfully reduced both attitude and angular velocity tracking errors while maintaining smooth spacecraft rotational motion.

The complete mission simulation validated the autonomous switching architecture between spacecraft operational modes.

3.6 STK Visualization

To validate the MATLAB simulations visually, spacecraft orbit propagation and attitude motion were reconstructed using AGI Systems Tool Kit (STK).

The generated visualization confirmed:

1. Correct orbital motion of both spacecraft,
2. Proper nadir and Sun-pointing orientations,
3. Communication alignment during GMO visibility windows,
4. Stable attitude transitions throughout mission operation.

3.7 Appendix: MATLAB Function Overview

Table 3.1: Main MATLAB Functions Used in the Project

Function Name	Description
`orbit_state()`	Computes spacecraft inertial position and velocity vectors
`orbitTheta()`	Computes orbital angle evolution
`orbit_frame_dcm()`	Generates Hill-frame DCM
`RsN_DCM()`	Returns Sun-pointing reference-frame DCM
`RnN_DCM()`	Returns nadir-pointing reference-frame DCM
`RcN_DCM()`	Returns GMO-pointing reference-frame DCM
`attitudeErrorEval()`	Computes tracking errors
`simulate_mode()`	Simulates spacecraft attitude dynamics
`getReferenceFrame()`	Performs automatic mission-mode selection

Conclusion

This project presented the complete development and implementation of an attitude dynamics and control framework for a nano-satellite orbiting Mars. Multiple mission pointing modes were developed, including Sun-pointing, nadir-pointing, and GMO communication-pointing configurations.

The spacecraft dynamics, reference-frame generation, attitude tracking evaluation, and closed-loop control architecture were successfully implemented in MATLAB and validated through numerical simulation.

The final integrated mission simulation demonstrated stable spacecraft attitude behavior while autonomously switching between operational mission modes according to orbital conditions.

Future improvements may include higher-fidelity environmental disturbance modeling, actuator limitations, sensor-noise modeling, and advanced nonlinear control strategies.